Microsoft Foundry Local

On-device AI you can ship inside your app

May 2026 — sourced from Microsoft Learn

Agenda

1. What Foundry Local is
2. Why on-device AI
3. Architecture in one slide
4. Hardware acceleration
5. Model lifecycle & catalog
6. Developer experience (SDK + CLI)
7. When to use it (and when not)
8. Get started

1. What is Foundry Local?

Foundry Local in one paragraph

- End-to-end **local AI runtime** your app ships with — not a cloud service
- Native **Core API** loaded **in-process** (`.dll` / `.so` / `.dylib`)
- SDKs for **C#, JavaScript, Python, Rust**
- Runtime adds only **~20 MB** to your application package
- Runs on **Windows, macOS (Apple Silicon), Linux**
- **No Azure subscription required**

2. Why on-device AI?

Motivation

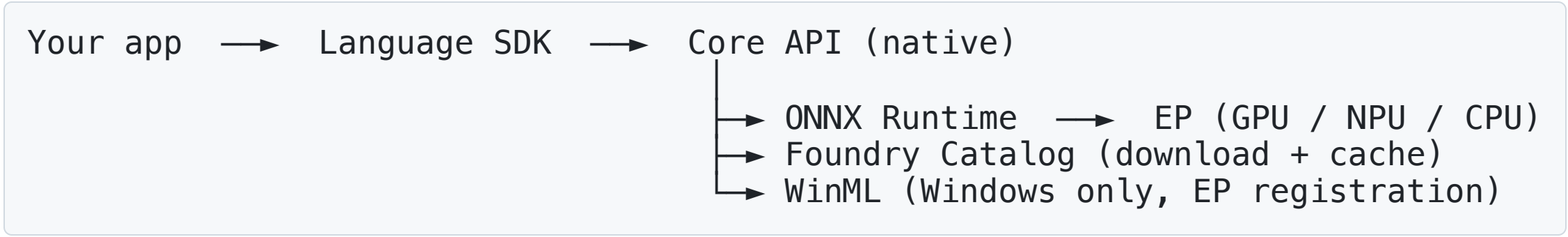
- **Privacy** — prompts and outputs never leave the device
- **Offline / limited connectivity** scenarios
- **Zero per-token cost** — no cloud inference bill
- **Low latency** — no network round-trip, instant first token
- **No backend to operate** — no servers, no scaling, no quota

3. Architecture

Components

- **Core API** — native library, loaded in-process by your app
- **Language SDKs** — `Microsoft.AI.Foundry.Local` (NuGet), `foundry-local-sdk` (npm / PyPI / crates.io)
- **ONNX Runtime** — the inference engine; graph optimization + execution providers
- **Foundry Catalog** — cloud-hosted registry of optimized models
- **WinML** (Windows) — sources & registers execution providers via OS / Windows Update
- **Optional REST API** — OpenAI-compatible HTTP endpoint inside your process

Request flow



- Direct **in-process function calls** — no HTTP overhead by default
- REST endpoint only when you need LangChain, Open WebUI, etc.

4. Hardware acceleration

Execution providers

Provider	Device	Platforms
NVIDIA CUDA	GPU	Windows, Linux
WebGPU (via Dawn)	GPU	Windows, Linux, macOS
AMD Vitis	NPU	Windows
Qualcomm	NPU	Windows
Intel OpenVINO	GPU	Windows
CPU	CPU	All — always available as fallback

- Foundry Local **auto-detects** hardware and picks the best EP
- On macOS: ONNX Runtime → WebGPU → **Dawn → Metal → Apple Silicon GPU**

5. Models & lifecycle

Curated catalog

- Chat: **GPT OSS, Qwen, DeepSeek, Mistral, Phi**
- Audio transcription: **Whisper**
- Every model **quantized & compressed** for on-device use
- **Versioned** — pin a version or auto-update
- Pre-compiled **per-hardware variants** (CPU / GPU / NPU)
- Or bring your own: compile Hugging Face models to ONNX

Lifecycle (in-process)

1. **Download** — pulled from the catalog on first use, cached on disk
2. **Load** — ONNX Runtime session + best EP selected for the device
3. **Inference** — sync or streaming chat completions
4. **Unload** — free memory; cached files stay on disk

6. Developer experience

OpenAI-compatible — minimal code change

```
import { FoundryLocalManager } from 'foundry-local-sdk';

const manager = FoundryLocalManager.create({ appName: 'my_app' });
await manager.downloadAndRegisterEps(() => {});

const model = await manager.catalog.getModel('qwen2.5-0.5b');
await model.download();
await model.load();

const chat = model.createChatClient();
const res  = await chat.completeChat([
  { role: 'user', content: 'Why is the sky blue?' }
]);

console.log(res.choices[0].message.content);
await model.unload();
```

Install

```
# JavaScript — Windows build integrates with Windows ML
npm install foundry-local-sdk-winml openai

# Cross-platform
npm install foundry-local-sdk
```

Other SDKs:

- **C#** — `Microsoft.AI.Foundry.Local` (NuGet)
- **Python** — `foundry-local-sdk` (PyPI)
- **Rust** — `foundry-local-sdk` (crates.io)

7. When to use it — and when not

Good fit

- Desktop / client apps embedding an LLM
- Privacy-sensitive workloads (healthcare, legal, on-device assistants)
- Field / offline scenarios
- Cost-sensitive features where per-token pricing doesn't work
- Real-time UX requiring sub-second first-token latency

Wrong fit

- **Multi-user server inference** — use vLLM or Triton instead
- General model experimentation — catalog is intentionally curated
- Very large frontier models — designed for hardware-constrained devices

8. Get started

Next steps

- Read **What is Foundry Local?** on Microsoft Learn
- Clone `github.com/microsoft/Foundry-Local` samples
- Pick an SDK (C# / JS / Python / Rust) and run the quickstart
- Try the **optional REST server** with LangChain or Open WebUI
- Evaluate a **catalog model** against your task; pin a version

Thank you

Questions?

Sources: Microsoft Learn — *What is Foundry Local?*, *Architecture overview*, *Get started*